

MASTER IMPLEMENTATION SPECIFICATION

Custom 3D Isometric Action RPG Engine

Autonomous Coding Agent Edition • Version 1.0

Purpose: This document is the canonical implementation contract for an AI coding agent tasked with building a bespoke, high-performance 3D isometric Action RPG engine and game framework without using a third-party game engine.

1. Project Mission and Non-Negotiable Constraints

Build a custom engine specifically optimised for a Diablo-style 3D isometric Action RPG. The target workload includes hundreds of simultaneously active enemies, projectiles, loot objects and visual effects, deterministic fixed-step gameplay, grid-aware navigation, data-driven content and an integrated developer toolchain.

Non-negotiable constraints:

- No Unity, Unreal Engine, Godot or other general-purpose game engine.
- Engine runtime and gameplay code must remain architecturally separated.
- Gameplay simulation must run on a fixed timestep.
- Runtime heap allocation inside performance-critical gameplay loops is prohibited unless explicitly profiled and justified.
- Core systems must favour contiguous storage, cache locality, batching and predictable ownership.
- Every milestone must compile, run and pass its completion gate before the next milestone begins.
- No pseudocode, empty implementations or TODO placeholders may be used to claim a milestone is complete.
- Third-party utility libraries are permitted only where explicitly approved by this specification. They must not replace ownership of the engine architecture.

2. Canonical Technology Stack

The project shall use one locked stack to eliminate architectural ambiguity.

Language: C++20.

Build system: CMake.

Graphics: Vulkan 1.3.

Shader language: GLSL compiled to SPIR-V.

Windowing/Input bootstrap: GLFW.

Vulkan loader: volk.

Mathematics: GLM.

Developer tooling UI: Dear ImGui.

Testing: Catch2 or GoogleTest, selected once during repository bootstrap and then used consistently.

Primary development target: 64-bit Linux desktop.

Secondary target: 64-bit Windows desktop.

Source control: Git.

The agent must not switch language, graphics API, build system or core stack without an explicit project-level decision.

3. Repository Architecture and Dependency Rules

/arpg-engine-root

■■■■ CMakeLists.txt

■■■■ README.md

■■■■ AGENTS.md

■■■■ /cmake

■■■■ /docs

■■■■ /core-engine

■ ■■■■ /include

■ ■■■■ /src

■ ■ ■■■■ /platform

■ ■ ■■■■ /core

■ ■ ■■■■ /memory

■ ■ ■■■■ /logging

■ ■ ■■■■ /ecs

■ ■ ■■■■ /renderer

■ ■ ■■■■ /assets

■ ■ ■■■■ /input

■ ■ ■■■■ /world

■ ■ ■■■■ /collision

■ ■ ■■■■ /navigation

■ ■ ■■■■ /audio

■ ■ ■■■■ /ui

■ ■■■■ /shaders

■■■■ /arpg-gameplay

■ ■■■■ /include

■ ■■■■ /src

■ ■■■■ /components

■ ■■■■ /systems

■ ■■■■ /combat

■ ■■■■ /abilities

■ ■■■■ /ai

■ ■■■■ /items

■ ■■■■ /inventory

■ ■■■■ /loot

■ ■■■■ /serialization

■■■■ /dev-tools

■ ■■■■ /editor

■ ■■■ /map-generator

■ ■■■ /ui-designer

■ ■■■ /asset-compiler

■■■ /tests

■ ■■■ /unit

■ ■■■ /integration

■ ■■■ /determinism

■ ■■■ /performance

■■■ /assets-source

■■■ /assets-compiled

■■■ /build

Dependency direction:

Platform/Core → Engine subsystems → Gameplay → Applications/Tools.

core-engine must never depend on arpg-gameplay.

Runtime code must never depend on dev-tools.

Tools may depend on core-engine and shared asset-format libraries.

Gameplay components must not contain renderer-specific Vulkan objects.

Renderer code must not contain ARPG combat or inventory rules.

4. Autonomous Coding Agent Operating Contract

The coding agent shall operate using the following loop for every milestone:

1. INSPECT: Read the repository, existing architecture, build files, tests and relevant documentation before modifying code.
2. PLAN: Produce or internally maintain a concrete implementation plan for the current milestone.
3. IMPLEMENT: Make the smallest coherent set of changes that fully implements the current task.
4. FORMAT/LINT: Apply the project's established formatting and static-analysis rules.
5. BUILD: Perform a clean or appropriate incremental build.
6. TEST: Run all tests relevant to the changed subsystem plus mandatory regression tests.
7. SMOKE TEST: Launch the appropriate executable when the milestone produces runtime behaviour.
8. VERIFY: Check acceptance criteria explicitly.
9. DOCUMENT: Update architecture or format documentation when contracts change.
10. CHECKPOINT: Create a logical Git commit if repository permissions and workflow permit.

The agent may work autonomously throughout a milestone. It must stop and request human input only when:

- A required architectural decision is genuinely unspecified and alternatives would create incompatible long-term designs.
- Required credentials, proprietary assets or unavailable external resources are needed.
- Existing project state conflicts materially with this specification.
- A destructive migration cannot be performed safely.

- Continuing would violate a non-negotiable constraint.

The agent must not ask for confirmation for routine implementation choices that can be safely inferred from this specification.

5. Global Engineering Standards

Warnings are treated seriously and new warnings should not be introduced. Debug builds must enable assertions and Vulkan validation layers where available. Release builds must disable expensive debug validation while retaining actionable error logging.

Ownership must be explicit. Prefer RAI for resource lifetime. Vulkan resource destruction order must be deterministic. Avoid raw owning pointers. Runtime systems must expose clear lifecycle phases: initialise, update where applicable, shutdown.

All externally stored binary formats must contain a magic identifier, format version and sufficient metadata for validation. Invalid or incompatible data must fail cleanly with a diagnostic rather than causing undefined behaviour.

Performance-critical code must avoid hidden allocations. Any unavoidable dynamic allocation in a hot path must be documented and measured.

All randomness affecting simulation must use explicit seeded pseudo-random number generators. Deterministic tests must be able to replay identical simulation inputs and obtain identical state hashes.

6. Runtime Loop and Determinism Contract

Simulation frequency: exactly 60 fixed updates per second using a fixed delta of 1/60 second represented consistently by the simulation layer.

Rendering is variable-rate and decoupled from simulation. An accumulator-based loop shall execute zero or more fixed simulation ticks before each render. Rendering may interpolate between previous and current simulation transforms.

The engine must define safeguards against the spiral of death, including a configurable maximum number of simulation catch-up steps per rendered frame.

Input events are collected by the platform layer and converted into engine action state. Simulation-relevant input is consumed at fixed-tick boundaries.

A headless simulation mode must eventually allow deterministic gameplay tests without initialising Vulkan or opening a window.

7. ECS Requirements

Entities use generational identifiers so stale entity references can be detected. Components are data-only structures. Component storage must be contiguous or chunk/archetype based. System iteration must avoid per-entity virtual dispatch.

Required operations include entity creation/destruction, component add/remove, component lookup, efficient multi-component queries and safe deferred structural changes during system iteration.

The initial performance target is stable simulation of at least 1,000 lightweight moving entities in the benchmark scene, with architecture designed to scale beyond this. Exact final budgets are established through profiling rather than unsupported assumptions.

8. Renderer Architecture

The Vulkan renderer must progress from a minimal primitive pipeline into a production-oriented architecture.

Required foundations:

- Vulkan instance, physical-device selection, logical device and queues.
- Swapchain creation and recreation.
- Command pools and command buffers.
- Synchronisation primitives and frames in flight.
- Depth buffering.
- GPU buffer and image abstractions with deterministic lifetime.
- Descriptor management.
- Shader module loading.
- Pipeline creation and caching strategy.
- Mesh and texture resources.
- Camera data.
- Batched or instanced rendering for repeated meshes.
- Frustum culling.
- Debug naming and validation integration.

Renderer-facing handles must prevent gameplay code from owning raw Vulkan objects. Resource upload and destruction must be coordinated so in-flight GPU work cannot access freed resources.

Later renderer milestones may add lighting, shadows, materials, particles and post-processing only after profiling and vertical-slice requirements justify them.

9. Asset Pipeline

Raw development assets live in assets-source. Runtime-ready assets live in assets-compiled. The runtime must consume compiled assets wherever practical rather than repeatedly parsing expensive authoring formats.

Each compiled asset type must have a versioned format and stable asset identifier. References between assets should use asset IDs or canonical virtual paths rather than fragile absolute filesystem paths.

The asset compiler must validate inputs, report actionable errors and avoid producing partially valid output. Incremental compilation should be supported when practical.

Required initial asset classes: shaders, meshes, textures, maps and UI layouts. Later classes include animation, audio, item definitions, abilities and effects.

10. World, Scene and Spatial Architecture

The engine requires an explicit world representation before substantial gameplay is added. A world owns ECS state and references loaded map/content resources.

The spatial layer must support efficient queries for nearby entities and collision candidates. Select a grid, spatial hash or other ARPG-appropriate structure based on measured workload. Do not perform $O(N^2)$ broad-phase entity collision checks in production gameplay.

Static map collision and dynamic entity collision must be distinguishable. Navigation walkability must have a defined relationship with static collision data.

11. Camera, Input, Picking and Movement

Implement an isometric ARPG camera using a configurable perspective or orthographic projection. Camera pitch, yaw, distance, zoom limits and tracking behaviour are data/configuration rather than unexplained constants.

Input must use named actions such as MoveCommand, PrimaryAbility, SecondaryAbility and Inventory rather than gameplay code polling raw platform keys directly.

Mouse picking converts screen coordinates into a world-space ray using the inverse view-projection matrix. Ground intersection and later collision picking must be testable independently.

Click-to-move issues navigation requests and movement follows resulting paths. Rendering interpolates simulation transforms rather than modifying authoritative simulation state.

12. Navigation

Initial navigation uses a 2D walkability grid and A*.

A* requirements:

- Explicit node coordinate representation.
- Deterministic tie-breaking.
- Configurable orthogonal/diagonal movement policy.
- Correct movement costs and admissible heuristic.
- Reusable search memory to minimise allocations.
- Unreachable-destination handling.
- Path simplification where appropriate.
- Unit tests for blocked, open, narrow and unreachable layouts.

The architecture must support multiple simultaneous agents. Path requests should be queueable and budgetable so a large spawn wave cannot create an uncontrolled single-frame CPU spike.

Dynamic obstacle handling and local avoidance are later extensions and must not be falsely claimed as implemented by basic static-grid A*.

13. Collision and Gameplay Queries

Provide gameplay-oriented collision and query primitives independently from rendering:

- Ray vs plane.
- Ray vs AABB.
- Point/circle or capsule-style 2D ground-plane overlap as appropriate.
- Nearby-entity queries.
- Static obstacle queries.
- Trigger volumes.

The project does not initially require a full general-purpose rigid-body physics engine. ARPG-specific movement and collision should remain deterministic and purpose-built unless later requirements demonstrate otherwise.

14. Combat and Ability Architecture

Combat must be data-driven and simulation-owned.

Core concepts:

- CombatStatsComponent.
- Health/resource state.
- Ability definitions.
- Cooldowns.
- Targeting rules.
- Damage events.
- Damage calculation pipeline.
- Projectiles.
- Area-of-effect queries.
- Status effects/modifiers.
- Death events.

Systems communicate through explicit events/queues or well-defined ECS state rather than tightly coupled direct calls. Rendering and VFX react to gameplay events but do not determine combat outcomes.

The first combat vertical slice must allow a player to move, target or aim an ability, damage enemies, kill them and receive a visible gameplay result.

15. Enemy AI and Spawning

Implement AI incrementally:

1. Idle.
2. Acquire target.
3. Navigate/chase.
4. Enter attack range.
5. Attack using the same ability/combat framework used by other actors.
6. Die and despawn or transition to corpse state.

Use lightweight state machines or data-oriented state representations. Avoid per-enemy heavyweight behaviour-tree machinery unless profiling and gameplay complexity justify it.

Spawner systems must support deterministic seeded waves for repeatable testing.

16. Items, Loot and Grid Inventory

Items are defined by data and referenced through stable item definition IDs. Runtime item instances hold instance-specific state separately from definitions.

Inventory uses a $W \times H$ occupancy grid. Multi-cell items have width and height.

Required operations:

ValidateFit(Item, X, Y)

InsertItem(Item, X, Y)

RemoveItem(ItemInstanceID)

MoveItem(ItemInstanceID, X, Y)

Operations must be atomic: failed moves or insertions must not corrupt occupancy state. Unit tests must cover boundaries, overlap, removal, relocation and malformed input.

Loot generation must be deterministic when supplied the same seed and inputs.

17. Runtime UI and UI Designer

The runtime UI system and UI designer are separate concerns.

The runtime must load compiled or validated UI layout data and render anchored elements across supported resolutions. Anchor behaviour, offsets, dimensions, visibility and texture references must have explicit semantics.

The WYSIWYG designer may use Dear ImGui. It must support selection, drag positioning, resizing, anchor selection, property editing, preview resolution and export.

Authoring format may be JSON. Runtime may consume a validated/compiled representation. UI schemas must include a schema version.

Initial HUD: health display, resource display, action quickbar and inventory window.

18. Procedural Dungeon Generator and .amap Format

The standalone generator shall support at least one deterministic algorithm, initially BSP or Cellular Automata, with a user-provided seed.

The generator must validate connectivity between required gameplay regions.

The .amap format must be formally documented. Minimum header:

- Magic bytes identifying AMAP.
- Format version.
- Width.
- Height.
- Tile encoding/version information.
- Payload size.
- Optional checksum if adopted.

Initial tile values:

0x00 Void/Wall

0x01 Walkable Floor

0x02 Player Spawn

0x03 Enemy Spawn

The loader must reject invalid magic, unsupported versions, impossible dimensions and truncated payloads.

Format evolution must be versioned. Never silently reinterpret incompatible map data.

19. Save and Load Architecture

Save games are not raw memory dumps. Persist stable gameplay identifiers and serialisable state using a versioned schema.

Save data must include a format version and support validation. The architecture must permit migration of older save versions where practical.

Transient renderer resources, pointers and ECS internal storage addresses must never be serialised as authoritative save data.

20. Animation, Audio and VFX

These systems are required for the production roadmap but should follow the core playable vertical slice.

Animation: skeleton/clip representation, animation state, playback, blending requirements and renderer skinning integration.

Audio: sound asset IDs, one-shot playback, looping, categories and spatial positioning where required.

VFX: event-driven effects for abilities, impacts, deaths and loot. High-volume effects should use pooling and GPU-friendly rendering where practical.

None of these systems may own authoritative combat outcomes.

21. Diagnostics, Profiling and Developer Features

Required developer facilities:

- Structured logging with severity levels.
- Assertions in debug builds.
- Frame timing.
- Fixed-tick timing.
- Per-system CPU timing.
- Entity counts.
- Draw-call and rendered-instance counts.
- GPU timing when renderer maturity permits.
- Memory allocator statistics.
- Debug overlay.
- Developer console or command interface where useful.

Optimisation decisions must be driven by measured profiles. The agent must not perform large speculative rewrites solely on the assumption that code is slow.

22. Testing Strategy

Unit tests cover pure algorithms and data structures: allocators, ECS operations, inventory, A*, binary formats and mathematical intersections.

Integration tests cover subsystem boundaries: map load into world, asset resolution, navigation-to-movement, combat-to-death and save/load round trips.

Determinism tests run identical seeded simulation inputs multiple times and compare stable state hashes.

Performance tests provide repeatable benchmark scenes. They are used to detect major regressions, while hardware-specific frame-rate numbers are recorded with environment information.

Headless tests must not require Vulkan where graphics output is irrelevant.

23. Performance Budgets and Benchmark Scenarios

Initial engineering targets, subject to measurement and refinement:

- Fixed simulation target: 60 Hz.
- Simulation budget: 16.67 ms maximum to maintain real-time fixed-step execution, with a preferred substantially lower normal-case cost to leave headroom.
- No routine heap allocation in established hot loops.
- Benchmark A: 1 player + 1,000 simple moving entities.
- Benchmark B: 500 enemies performing lightweight target/chase behaviour.
- Benchmark C: sustained projectile and damage-event workload.
- Benchmark D: repeated path requests across a representative dungeon.
- Benchmark E: inventory stress and repeated item moves.

A milestone is not considered performant merely because a trivial scene reaches a high frame rate.

24. Milestone Roadmap

M0 – Repository and Build Bootstrap

Deliver: reproducible CMake build, dependency setup, engine/game/tool/test targets, CI-ready test command.

Gate: clean configure/build/test from documented commands.

M1 – Platform Layer and Fixed Game Loop

Deliver: window, input/event pump, timer, fixed 60 Hz simulation loop, variable render loop.

Gate: timing tests and runnable empty application.

M2 – Memory, Logging and Diagnostics Foundation

Deliver: arena/pool foundations, assertions, logging, frame/tick metrics.

Gate: allocator tests and debug diagnostics.

M3 – ECS

Deliver: entity lifecycle, generational IDs, component storage, queries and deferred structural changes.

Gate: correctness tests and entity iteration benchmark.

M4 – Vulkan Renderer Foundation

Deliver: swapchain, synchronisation, depth, primitive mesh and camera.

Gate: stable primitive rendering and resize/swapchain recreation.

M5 – Asset Pipeline

Deliver: asset IDs, shader/texture/mesh pipeline and compiler foundations.

Gate: compiled assets load through engine resource interfaces.

M6 – World and Scene Foundation

Deliver: world ownership, map representation and spatial-query foundation.

Gate: test map loads into a valid runtime world.

M7 – Camera, Input and Picking

Deliver: isometric camera, action mapping and mouse-to-world picking.

Gate: click location is visualised and mathematically tested.

M8 – Navigation and Movement

Deliver: A*, click-to-move and multi-request scheduling foundation.

Gate: player reliably navigates obstacle test maps.

M9 – Collision and Spatial Queries

Deliver: required query primitives and static/dynamic candidate lookup.

Gate: movement and targeting queries pass integration tests.

M10 – Combat Vertical Slice

Deliver: player ability, enemy health, damage, projectile/AOE as selected, death.

Gate: playable combat loop.

M11 – Enemy AI and Scale

Deliver: target acquisition, chase, attack, spawning.

Gate: benchmark enemy encounter remains within established simulation budget.

M12 – Items, Loot and Inventory

Deliver: definitions, instances, deterministic loot and grid inventory.

Gate: loot can drop, be collected and manipulated without grid corruption.

M13 – Runtime UI

Deliver: HUD, quickbar and inventory UI.

Gate: playable loop communicates health/resources and supports inventory interaction.

M14 – Procedural Dungeon Generation

Deliver: deterministic generator and versioned .amap export/load.

Gate: generated connected dungeon loads and is playable.

M15 – Editor and Content Tools

Deliver: map-generation UI, UI layout designer and content validation.

Gate: tool-generated content is consumed by runtime without manual file editing.

M16 – Save/Load

Deliver: versioned save format and round-trip restoration.

Gate: representative game state saves and restores correctly.

M17 – Animation, Audio and VFX

Deliver: production presentation foundations.

Gate: vertical slice has integrated animation/audio/effects without gameplay authority leakage.

M18 – Optimisation and Production Hardening

Deliver: profiling-led improvements, stress testing, failure handling and documentation.

Gate: agreed benchmark targets pass and no known critical correctness defects remain.

25. Milestone Definition of Done

Every milestone must satisfy all applicable conditions:

- Code compiles in supported development configuration.
- New automated tests pass.
- Existing regression tests pass.
- No known crash occurs in the documented smoke test.
- No placeholder implementation is represented as complete.
- Public subsystem contracts are documented.
- Persisted format changes are versioned.
- Performance-sensitive changes are measured where applicable.
- The milestone's explicit acceptance gate passes.

If any condition fails, the agent remains in the current milestone.

26. Agent Change Discipline

Before changing an existing interface, search all call sites and tests. Prefer incremental compatible changes where reasonable.

Do not rewrite working subsystems merely to match personal stylistic preference. Refactor when required for correctness, architecture, maintainability or measured performance.

Do not silently delete tests to make a build pass. A test may be changed only when the underlying contract intentionally changes, and the reason must be clear.

Do not suppress compiler, validation or static-analysis warnings without understanding the cause.

Do not add dependencies casually. Any new dependency must have a specific purpose, appropriate licence and clear advantage over implementing the required narrow functionality internally.

27. First Playable Vertical Slice

The first major product objective is not a feature-complete engine. It is a coherent playable proof of the architecture.

The vertical slice must contain:

- A loaded 3D dungeon/map.
- Isometric player camera.
- Click-to-move.
- Obstacle-aware navigation.

- At least one enemy archetype.
- Enemy chase and attack behaviour.
- At least one player combat ability.
- Damage and death.
- Loot drop.
- Grid inventory pickup and placement.
- Health/resource HUD.
- Basic debug/performance overlay.

This slice is the primary architectural validation point. Advanced rendering features, elaborate editors and broad content systems must not delay it unless they are prerequisites.

28. Final Directive to the Coding Agent

Treat this document as the canonical project implementation contract.

Begin at M0 unless repository inspection proves earlier milestones are already complete. When code already exists, audit it against the milestone gates rather than blindly recreating it.

Work one milestone at a time. Work autonomously within that milestone. Build and test continuously. Fix regressions before advancing. Preserve strict dependency direction. Keep gameplay simulation independent from rendering. Keep persisted formats versioned. Measure performance instead of guessing.

When reporting progress, state:

1. Milestone being worked on.
2. Changes implemented.
3. Files/modules materially changed.
4. Build command and result.
5. Tests executed and result.
6. Smoke test performed.
7. Acceptance criteria status.
8. Known limitations or blockers.
9. Exact next step.

Never claim functionality that has not been implemented and verified.

The objective is not to create a generic game engine. The objective is to create the smallest robust, high-performance custom engine and toolchain necessary to build the intended 3D isometric Action RPG successfully.

29. Critical Edge-Case and Runtime Safety Directives

The following requirements are mandatory safeguards for their respective milestones. These requirements define safety behaviour; implementations may use more advanced synchronisation or data structures provided they preserve the stated guarantees.

29.1 Vulkan Surface, Resize and Minimisation Handling (Milestone 4)

- Detect framebuffer resize events and handle `VK_ERROR_OUT_OF_DATE_KHR` and `VK_SUBOPTIMAL_KHR` appropriately.
- Before creating or recreating a swapchain, obtain the current drawable framebuffer extent.
- A zero-width or zero-height framebuffer must never be used to create swapchain resources.
- When the framebuffer extent is zero, such as while the application is minimised, suspend rendering and presentation until a valid non-zero drawable extent becomes available.
- The application must avoid busy-waiting while minimised. GLFW event-waiting or another appropriate throttled wait mechanism should be used.
- The simulation behaviour while minimised must be explicitly defined by application policy. Rendering suspension must not accidentally alter simulation determinism.
- Swapchain-dependent resources, including image views and any extent- or format-dependent render resources, must be recreated safely.
- Resources must not be destroyed while they remain in use by GPU or presentation operations.
- The initial implementation may use `vkDeviceWaitIdle` during infrequent swapchain recreation for correctness and simplicity. A later optimisation may replace this with finer-grained fence/presentation synchronisation where profiling or production requirements justify it.
- Swapchain recreation must be tested for repeated resizing, minimisation/restoration and movement between displays where practical.

29.2 Fixed-Tick Input Buffering and Deterministic Input Commands (Milestones 1 and 7)

- Raw platform input must not be consumed directly as authoritative gameplay input inside individual fixed-update systems.
- Architecture: GLFW Events → Engine Input Buffer → Tick Input Snapshot / Commands → Gameplay Simulation.
- GLFW callbacks or equivalent event mechanisms must capture discrete transitions such as key presses, key releases, mouse-button presses, mouse-button releases and scroll events so short-lived transitions cannot be lost between simulation ticks.
- Persistent input state, such as held keys or buttons, must be maintained separately from discrete edge events.
- Continuous values, including cursor position and analogue controller axes, must be sampled into a coherent input snapshot for simulation consumption.
- At each fixed simulation tick, the input system must produce a stable tick-level `InputSnapshot` and/or gameplay command set.
- Each discrete gameplay command consumed by deterministic simulation must be associated with a simulation tick index.
- Deterministic replay and regression testing must record and replay tick-indexed gameplay commands rather than relying solely on operating-system event timestamps.
- Platform timestamps may be retained for ordering, latency measurement or diagnostics but must not be assumed to provide cross-run deterministic timing.
- Input must be consumed exactly once where event semantics require one-shot behaviour.
- The implementation must not introduce atomic or lock-based synchronisation unless input crosses thread boundaries. If platform polling and simulation execute on the same thread, a non-concurrent queue or buffer is preferred.
- If input is transferred between threads, the selected synchronisation mechanism must provide explicit ownership and race-free behaviour.
- Tests must cover: press and release occurring between two simulation ticks; multiple fixed ticks during one rendered frame; render rates substantially higher and temporarily lower than 60 Hz; one-shot commands executing

exactly once; and deterministic playback of a recorded tick-indexed input sequence.

29.3 Spatial Partition and Map Boundary Safety (Milestones 6 and 9)

- No world coordinate may be converted into an array-backed spatial or map index without explicit validation.
- Array-backed grid access must validate signed grid coordinates before indexing storage.
- Negative coordinates and coordinates equal to or greater than grid dimensions must never result in an out-of-bounds read or write.
- Conversion from signed coordinates to unsigned indices must occur only after successful bounds validation.
- Spatial queries extending partially outside a bounded map must safely ignore or clip the queried cell range without accessing invalid storage.
- The spatial partition implementation must clearly distinguish between bounded map grids and spatial structures, such as hash grids, that may legitimately represent negative or unbounded coordinates.
- Navigation-map boundaries, collision-world boundaries and spatial-partition boundaries must not be assumed to be identical unless explicitly defined as such.
- Gameplay movement outside the valid playable world must follow an explicit policy defined by the owning gameplay/world system. Valid policies may include collision rejection, boundary resolution, clamping, despawning or world transition.
- The spatial indexing layer must not silently clamp authoritative entity positions as a substitute for correct gameplay collision handling.
- Knockback, teleportation, spawning and high-velocity movement must all validate destination positions before bounded map access.
- Tests must include: negative world and grid coordinates; exact minimum and maximum boundaries; one cell beyond every boundary; large invalid coordinates; queries overlapping map edges; knockback across a world boundary; teleportation to an invalid location; malformed map dimensions; and overflow-resistant index calculations.

The overriding requirement is that malformed input, unusual gameplay movement or window-system state must result in controlled behaviour and diagnostics, never memory corruption, invalid GPU-resource usage or undefined behaviour.